

Modelagem de rede hidráulica a partir de álgebra linear computacional

Disciplina: SME0602 — Motores Numéricos para Simulação em Engenharia

Professor: Roberto F. Ausas

Grupo 3:

- Beatriz Cosimatti
- Cecilia Queiroz
- Gabriel Zago
- Matheus Buzzon
- Pedro Vale
- Victor Silva

Estrutura dos arquivos

— data_structures.py	-> Gera o grafo para modelar a rede hidráulica
— env.py	-> Armazena parâmetros de configuração do sistema
— hydraulics.py	-> Física e álgebra linear da rede hidráulica
— plotting.py	-> Visualização de resultados
— app.py	-> Executa a simulação

env.py

- Contém o dicionário CONFIG que centraliza todos os parâmetros físicos e é passado para as classes de simulação com o intuito de evitar hard-coding e tornar o código mais dinâmico.

```
CONFIG = {
    "N_INLET": 0,
    "INLET_FLOW": 1.0e-7,
    "INLET_PRESSURE": 100,
    "INLET_FLOW_DICT": {"0": 1.0e-7, "175": 1.0e-6},
    "INLET_PRESSURE_DICT": {"0": 1.0e-7, "175": 1.0e-6},
    "INLET_FLOW_SIN_DICT": {"N_INLET": 0, "A": 0.1, "omega": 3, "theta": 0, "B": 1},
    "INLET_FLOW_COS_DICT": {"N_INLET": 175, "A": 0.01, "omega": 4, "theta": 0, "B": 0.1},

    "N_OUTLET": 5,
    "OUTLET": 0,

    "PIPE_AREA": 0.00000025,
    "VISCOSITY": 0.001,
    "TIME_ANALYSIS": [0, 10, 1000]
}
```

hydraulics.py – Classe Hydraulics

```
class Hydraulics():
    def __init__(self, conec, Xno, config):
        self.conec = conec
        self.Xno = Xno

        self.num_nodes = np.max(conec) + 1
        self.num_pipes = np.shape(conec)[0]

        self.node_outlet = config["N_OUTLET"]
        self.node_inlet = config["N_INLET"]
        self.inlet = config["INLET_FLOW"]
        self.outlet = config["OUTLET"]
        self.pipe_area = config["PIPE_AREA"]
        self.viscosity = config["VISCOSITY"]

        self.results = {'P': None, 'Q': None, 'W': None}
```

A inicialização da classe Hydraulics recebe:

- conec (da função GeraGrafo)
- Xno (da função GeraGrafo)
- config (do env.py)

hydraulics.py – Classe Hydraulics

Métodos similares aos outros projetos:

Método	Função	Return
calculate_conductancy	Calcula a condutância hidráulica de cada cano usando a lei do escoamento laminar em dutos	Vetor C com as condutâncias de cada cano
assembly	Monta a matriz de condutâncias global A da rede, que representa as relações de pressão entre todos os nós.	Matriz A
solve_network	Impõe as condições de contorno e resolve o sistema linear para encontrar as pressões em cada nó.	Vetor p com as pressões em cada nó
calculate_flow_rate_and_potency	Com as pressões calculadas, determina a vazão em cada cano e a potência dissipada pela rede.	Vetor Q : vazões em cada cano Float W : Potência dissipada

hydraulics.py – Classe Hydraulics

Método run: Executa toda a simulação e exibe os resultados tanto no terminal, quanto os gráficos.

```
def run(self, print_info, plot):

    self.calculate_flow_rate_and_potency()

    if print_info:
        print(f"Resultados para classe: {self.__class__.__name__}")
        print(f"Solução das pressões em cada nó: {self.results['P']}")
        print(f"Solução das vazões em cada cano: {self.results['Q']}")
        print(f"Solução da Potência dissipada pelo sistema: {self.results['W']}\n\n")

    if plot:
        PlotaRede(self.conec, 1000*self.Xno, self.results['P'], self.results['Q'])
        plt.show()
```

hydraulics.py – Outros casos

Para encontrar as soluções dos outros casos além da injeção de vazão no nó de inlet e a pressão fixada no nó de outlet, decidimos usar o conceito de **herança de classe**.

```
class Hydraulics_p1(Hydraulics): ...  
  
class Hydraulics_p2(Hydraulics): ...  
  
class Hydraulics_p3(Hydraulics): ...  
  
class Hydraulics_p4(Hydraulics): ...  
  
class Hydraulics_p5(Hydraulics): ...  
  
class Hydraulics_p6(Hydraulics): ...
```

Essa técnica permite reutilizar facilmente os métodos que já foram construídos na classe Hydraulics, e apenas alterar os pedaços relevantes para os novos problemas.

hydraulics.py – Hydraulics_p1

Em vez de um único nó de injeção, recebe um dicionário com vários nós e suas respectivas vazões impostas.

O sistema linear continua sendo $\tilde{A}^*p = b$.

A única mudança é a forma como o vetor b é construído:

Para cada (nó_k, Q_k) em INLET_FLOW_DICT:
 $b[\text{nó}_k] = Q_k$

hydraulics.py – Hydraulics_p2

Generaliza a condição de saída: em vez de um único nó cuja pressão é definida, recebe um dicionário com vários nós e suas respectivas pressões impostas. O sistema linear continua sendo $\tilde{A}^*p = b$.

As únicas mudanças são a forma como a matriz $\tilde{A}$ e o vetor b são construídos:

Para cada (nó_k, P_k) em INLET_PRESSURE_DICT:

$\tilde{A}[\text{nó_k}, :] = 0$	# zera a equação de conservação do nó
$\tilde{A}[\text{nó_k}, \text{nó_k}] = 1$	# exceto na diagonal
$b[\text{nó_k}] = P_k$	# impõe a pressão externa no nó

hydraulics.py – Hydraulics_p3

O objetivo deste problema é resolver o sistema quando não se sabe a vazão de entrada, mas sabe-se a pressão no nó de entrada e no de saída.

As únicas mudanças são a forma como a matriz $\tilde{A}$ e o vetor b são construídos:

```

linha_inlet =  $\tilde{A}$ [node_inlet, :]           # preserva-se a linha do inlet
 $\tilde{A}$ [node_inlet, :] = 0                   # zera a equação de conservação do nó
 $\tilde{A}$ [node_inlet, , node_inlet, ] = 1      # exceto na diagonal
b[node_inlet, ] = P_INLET                # impõe a pressão externa no nó de entrada

 $\tilde{A}$ [node_outlet, :] = 0                  # zera a equação de conservação do nó
 $\tilde{A}$ [node_outlet, , node_outlet, ] = 1    # exceto na diagonal
b[node_outlet, ] = P_OUTLET              # impõe a pressão externa no nó de saída

pressures = np.linalg.solve( $\tilde{A}$ , b)      # Resolve-se o sistema normalmente

vazão_inlet = np.dot(linha_inlet, pressures)

```

hydraulics.py – Hydraulics_p4

Agora, a vazão de entrada em um ponto é uma função trigonométrica do tempo:

$$Q_{in}(t) = B \cdot \text{sen}(\omega t + \theta) + C$$

Portanto, o sistema linear que deveríamos resolver seria:

$$\tilde{A} * p(t) = b(t)$$

Para evitar calcular uma sistema linear para cada t no nosso intervalo de análise, gastando muitos recursos computacionais, usamos a propriedade linear do sistema, a qual permite que nós separássemos o vetor $b(t)$ como:

$$b(t) = b_B * \text{sen}(\omega t + \theta) + b_C$$

hydraulics.py – Hydraulics_p4

```

b_B[node_inlet] = B           # impõe a constante B
b_C[node_inlet] = C           # impõe a constante C

p_B = np.linalg.solve(Ã, b_B) # resolve o sistema para b_B
p_C = np.linalg.solve(Ã, b_C) # resolve o sistema para b_C

```

Por fim, podemos reconstruir $p(t)$ como:

$$p(t) = p_B^* \sin(\omega t + \theta) + p_C$$

hydraulics.py – Hydraulics_p5

Agora, a vazão de entrada no nó 0 continua sendo a função de antes, mas vamos injetar outra vazão com outra função trigonométrica no nó 175:

$$\begin{aligned} Q_{in_0}(t) &= B \cdot \text{sen}(\omega_1 t + \theta_1) + C \\ Q_{in_175}(t) &= D \cdot \text{cos}(\omega_2 t + \theta_2) + E \end{aligned}$$

Seguindo a mesma estratégia de antes, iremos calcular agora apenas 4 sistemas lineares, um para b_B, b_C, b_D e b_E e por fim iremos reconstituir p(t).

hydraulics.py – Hydraulics_p5

```

b_B[node_inlet_sin] = B           # impõe a constante B
b_C[node_inlet_sin] = C           # impõe a constante C
b_D[node_inlet_cos] = D           # impõe a constante B
b_E[node_inlet_cos] = E           # impõe a constante C

p_B = np.linalg.solve(Ã_0, b_B)   # resolve o sistema para b_B
p_C = np.linalg.solve(Ã_0, b_C)   # resolve o sistema para b_C
p_D = np.linalg.solve(Ã_175, b_D) # resolve o sistema para b_D
p_E = np.linalg.solve(Ã_175, b_E) # resolve o sistema para b_E

```

Por fim, podemos reconstruir $p(t)$ como:

$$p(t) = p_B * \text{sen}(\omega_1 t + \theta_1) + p_C + p_D * \cos(\omega_2 t + \theta_2) + p_E$$

hydraulics.py – Hydraulics_p6

Agora, a vazão no inlet e a pressão no outlet são constantes, mas a viscosidade do fluido é uma função do temperatura, que por sua vez é função do tempo:

$$T(t) = 20 + 0.9t^2$$

$$\mu(T) = \frac{1}{1 + 0.03368 \cdot T + 0.00221 \cdot T^2}$$

Para economizar recursos computacionais, calculamos apenas uma vez para a viscosidade constante usada nos outros problemas e pela classe Hydraulics, e depois multiplicamos por um fator de escala para encontrar as pressões naquele tempo de análise.

hydraulics.py – Hydraulics_p6

```
def find_max_pressures_over_time(self):
    # Resolvemos o sistema uma vez para a viscosidade de referência
    # (self.viscosity) e vazão de entrada (self.inlet).
    base_pressures = self.solve_network()

    time_start = self.time[0]
    time_end = self.time[1]
    increments = self.time[2]

    time_array = np.linspace(time_start, time_end, increments)
    max_pressures = []

    # Aproveitamos a linearidade do sistema
    for t in time_array:
        T = self.temperature(t)
        mu_atual = self.viscosity_t(T)

        # O fator de escala é a razão entre a viscosidade atual e a viscosidade base usada no solve_network
        fator_escala = mu_atual / self.viscosity

        pressures_in_t = base_pressures * fator_escala
```


hydraulics.py – Análise de complexidade

```
def complexity_analysis(HydraulicClass: Hydraulics, print_info, plot):
    levels_list = [1, 2, 3, 4] # os levels de análise
    results = []

    for level in levels_list:
        tempos_assembly = [] # tempo de montagem da matriz
        tempos_solve = [] # tempo de resolucao

        for _ in range(10): # repete 10 vezes - para tirar uma media depois
            # medindo tempo de montagem da matriz
            start = time.time()
            HydraulicClass.assembly()
            tempos_assembly.append(time.time() - start)

            # medindo tempo de resolucao do sistema linear
            start = time.time()
            HydraulicClass.solve_network()
            tempos_solve.append(time.time() - start)
```

Função que recebe uma instância de classe hidráulica e calcula o tempo de execução para cada nível

app.py – Centralização da execução

```
def main():
    config = env.CONFIG

    Xno, conec = GeraGrafo(levels=4)
    mm_to_m = 0.001
    Xno = Xno * mm_to_m

    test = Hydraulics(conec, Xno, config)
    test.run(print_info = True, plot = True)
    complexity_analysis(test, print_info = True, plot = True)

    test_p1 = Hydraulics_p1(conec, Xno, config)
    test_p1.run(print_info = True, plot = True)
    complexity_analysis(test_p1, print_info = True, plot = True)

    test_p2 = Hydraulics_p2(conec, Xno, config)
    test_p2.run(print_info = True, plot = True)
    complexity_analysis(test_p2, print_info = True, plot = True)

    # continua para as outras classes ...
```

Agradecemos pela atenção!